Idempotency

Idempotency is a property of an API operation where **making the same request multiple times produces the same result as making it once**. It is an important concept in REST APIs, including those built with **FastAPI**.

Why is Idempotency Important?

Suppose a client sends a request to create an order, but the network times out before receiving a response. The client retries the request.

- **Without idempotency:** Two orders may be created.
- **With idempotency:** Only one order is created, regardless of how many retries occur.

This prevents duplicate transactions, payments, or records.

HTTP Methods and Idempotency

HTTP Method	Idempotent?	Explanation
GET	✅ Yes	Retrieves data without changing it.
PUT	✅ Yes	Replaces an entire resource. Repeating it has the same effect.
DELETE	✅ Yes	Deletes a resource. After the first delete, subsequent deletes don't change the outcome.
POST	❌ No	Usually creates a new resource each time.
PATCH	Usually No	Applies partial updates that may not produce the same result on repeated calls.

Example 1: Non-Idempotent POST

```
from fastapi import FastAPI

app = FastAPI()

orders = []

@app.post("/orders")
def create_order(order: dict):
    orders.append(order)
    return {"message": "Order Created", "count": len(orders)}
```

Request

```
POST /orders
{
   "item":"Laptop"
}
```

If sent three times:

```
orders =
[
  {"item":"Laptop"},
  {"item":"Laptop"},
  {"item":"Laptop"}
]
```

Three different orders are created.

Example 2: Idempotent PUT

```
from fastapi import FastAPI

app = FastAPI()

users = {}
```

```
@app.put("/users/{id}")
def update_user(id: int, user: dict):
    users[id] = user
    return users[id]
```

Calling

```
PUT /users/1
{
  "name":"Murthy"
}
```

10 times still results in

```
{
  1:{
    "name":"Murthy"
  }
}
```

The resource remains the same.

Making POST Idempotent Using an Idempotency Key

Many payment gateways (such as Stripe) use an **Idempotency-Key**.

Client sends:

POST /payments

Headers:
Idempotency-Key: abc123

Server logic:

1. Check if key exists.
2. If yes, return the previous response.
3. Otherwise, process the request.

4. Store the response using the key.
5. Return the response.

FastAPI Example

```
from fastapi import FastAPI, Header
import uuid

app = FastAPI()

processed = {}

@app.post("/payments")
def make_payment(
    amount: int,
    idempotency_key: str = Header(...)
):

    if idempotency_key in processed:
        return processed[idempotency_key]

    payment = {
        "payment_id": str(uuid.uuid4()),
        "amount": amount,
        "status": "Success"
    }

    processed[idempotency_key] = payment

    return payment
```

First Request

```
POST /payments

Headers:
```

Idempotency-Key: xyz123

Body:
amount=1000

Response


```
{
   "payment_id":"b91d...",
   "amount":1000,
   "status":"Success"
}
```

Retry (Same Key)

POST /payments

Headers:
Idempotency-Key: xyz123

Response


```
{
   "payment_id":"b91d...",
   "amount":1000,
   "status":"Success"
}
```


Notice the **same payment ID** is returned. A second payment is **not** created.

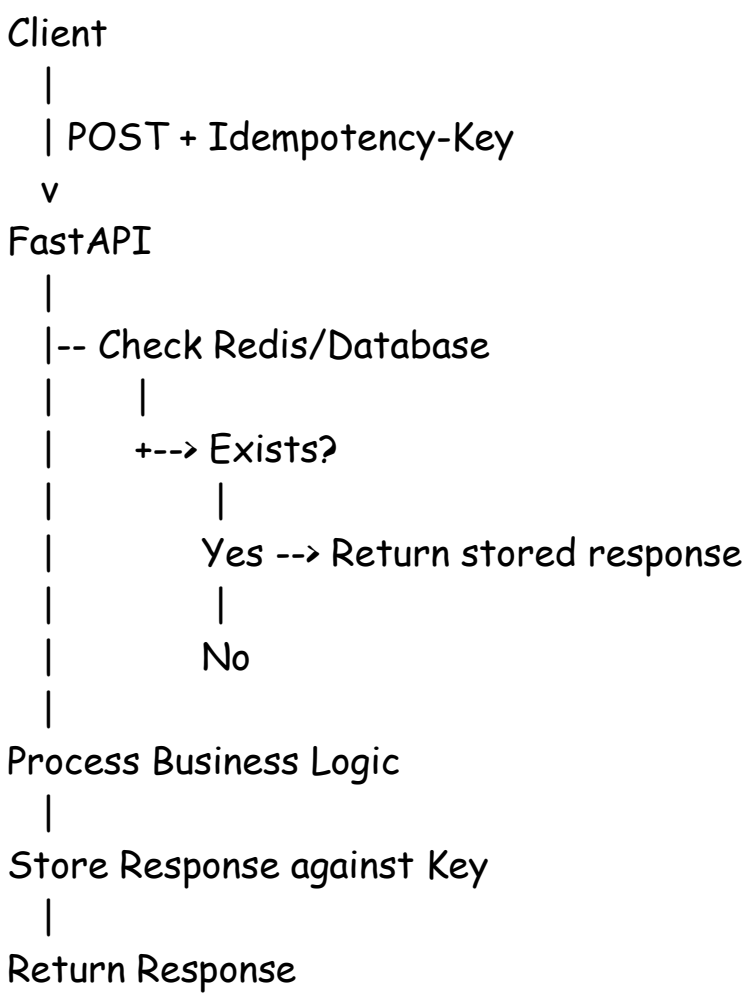
Production Implementation

Instead of storing keys in memory, use a persistent or distributed store such as:

- **Redis** (fast, supports expiration/TTL)
- PostgreSQL
- MySQL

- MongoDB

Typical flow:



Common Use Cases

- Payment processing
- Order creation
- Ticket booking
- Flight reservations
- Bank transfers
- Invoice generation
- Subscription renewals
- Inventory updates triggered by retries

Benefits

- Prevents duplicate requests from creating duplicate data.
- Handles network retries safely.
- Improves API reliability and resilience.
- Ensures consistent behavior in distributed systems.
- Simplifies client retry logic.